

# Galaaz 2.0: Ruby meets real GNU R (again)

NewBridge, JRuby or CRuby, CRAN/Bioconductor, and Arrow-shaped data paths

Rodrigo Botafogo

2026

## Contents

|          |                                                  |          |
|----------|--------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                              | <b>1</b> |
| <b>2</b> | <b>What changed</b>                              | <b>2</b> |
| <b>3</b> | <b>NewBridge in one picture</b>                  | <b>2</b> |
| <b>4</b> | <b>Either Ruby engine</b>                        | <b>3</b> |
| <b>5</b> | <b>Multiple R versions via containers</b>        | <b>3</b> |
| <b>6</b> | <b>Moving data: Apache Arrow</b>                 | <b>4</b> |
| 6.1      | Stage B in plain words . . . . .                 | 4        |
| 6.2      | Stage B sketch (B1 ingest + B2 export) . . . . . | 5        |
| 6.3      | Stage A (still useful) . . . . .                 | 6        |
| <b>7</b> | <b>Keeping the bridge free</b>                   | <b>7</b> |
| <b>8</b> | <b>What stayed the same</b>                      | <b>7</b> |
| <b>9</b> | <b>Conclusion</b>                                | <b>8</b> |
| 9.1      | Further reading . . . . .                        | 8        |

## 1 Introduction

Ruby is excellent for web apps, orchestration, and expressive object models. It has never matched **R** for statistics, Bioconductor pipelines, or the depth of CRAN. **Python** closed a similar gap with NumPy, pandas, and a huge scientific ecosystem. For Ruby, the practical answer is not to reimplement that world — it is to **drive real R** from idiomatic Ruby.

**Galaaz** is that coupling. An earlier line of work ran on Oracle’s **GraalVM** with **TruffleRuby** and **FastR** in one JVM. **Galaaz 2.0** is a different architecture: **JRuby** or **CRuby** talk

to **standard GNU R** over a process bridge we call **NewBridge**. You keep CRAN and Bioconductor (including compiled packages), and you keep a normal Ruby toolchain.

This post is the 2.0 story: what changed, how the bridge works, how to pin R versions in containers, how data moves (including **Apache Arrow**), and how to keep the bridge free for long R jobs. Deep API detail lives in the Galaaz Manual; here we stay at blog length with a few runnable sketches.

## 2 What changed

|              | Historical stack                                                   | Galaaz 2.0                                               |
|--------------|--------------------------------------------------------------------|----------------------------------------------------------|
| Ruby         | TruffleRuby (Graal)                                                | <b>JRuby</b> or <b>CRuby</b>                             |
| R            | FastR (subset of GNU R)                                            | <b>GNU R</b> (CRAN / Bioconductor)                       |
| Coupling     | Same-JVM interop                                                   | <b>NewBridge</b> (separate processes)                    |
| Install      | Special Graal distribution                                         | <code>gem install</code> + compile gatekeeper            |
| Legacy tools | <code>grun</code> , <code>polyglot</code> <code>gknit-draft</code> | <code>bin/galaaz-ruby</code> ,<br><code>bin/gknit</code> |

Ideas from the older articles still matter — ggplot layers, dplyr pipes, gKnit literate docs — but the **engine** underneath is different. If you land on a 2018 “Ruby + R on GraalVM” post, treat the plots and narrative as inspiration and this post (plus the manual) as the current stack.

## 3 NewBridge in one picture

Galaaz 2.0 does **not** embed Renjin or FastR. Roughly:

1. Your **Ruby** process (JRuby or CRuby) loads Galaaz.
2. A native **gatekeeper** (`ext/new_bridge`, Rcpp / C++) mediates the protocol.
3. A **GNU R** process evaluates calls, holds large objects, and runs packages.

Ruby stays Ruby: `R.mean`, `E.aes`, method chaining, classes and modules. R stays R: the same `lm`, `DESeq2`, or `ggplot2` you would run in RStudio. Large panels should usually **stay in R**; Ruby orchestrates and pulls small results (Remote Control). Opt in to unbox scalars when you need a plain Ruby number (`>> 0` for length-1 vectors is the common 2.0 idiom).

A minimal session looks like this:

```
require 'galaaz'

puts R.sum(R.c(1, 2, 3, 4, 5))
mtcars = ~R[:mtcars]
puts mtcars.dim
puts R.summary(mtcars.mpg)
```

```
## [1] 15
## [1] 32 11
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    10.40   15.43   19.20   20.09   22.80   33.90
```

Callbacks still exist: Ruby procs can be registered so R code can call back into Ruby when a package expects an R function. That path is documented in the manual; the important product point is that **real GNU R** is on the other end of the wire.

## 4 Either Ruby engine

NewBridge is tested on both:

- **JRuby** (e.g. 10.x + JDK 21) — real OS threads, JVM ecosystem, natural fit for Rails under load.
- **CRuby** (e.g. 3.3+) — MRI gem ecosystem, familiar for many Ruby shops.

The public API is the same. Launchers honor `GALAAZ_RUBY`. Use `bin/galaaz-ruby`, or `bin/galaaz-jruby` to force JRuby. Prefer **JRuby** when you want parallel Ruby threads feeding R; prefer **CRuby** when your app and gems already live there. Arrow helpers need engine-specific native bits (Java Arrow + `JAVA_OPTS` nio opens on JRuby; Arrow GLib + `red-arrow` on CRuby) — see the manual’s Arrow section.

```
# From a checkout or installed layout:
bin/galaaz-ruby -e \
  'require "galaaz"; puts R.R__version[["version.string"]]'
GALAAZ_RUBY=jruby bin/galaaz-ruby -e \
  'require "galaaz"; puts R.pi'
```

## 5 Multiple R versions via containers

Because R is a **separate process**, the Ruby gem does not hard-wire one R build. Whatever R / Rscript is on PATH (and can load the gatekeeper) is the R you drive. That makes **containers** a clean way to pin versions:

- Image A: Ubuntu + **R 4.3** + system libs for your Bioconductor set.
- Image B: same app code + **R 4.4** (or a rocker image) for comparison.
- Your Galaaz gem and Ruby code stay the same; only the image’s R changes.

Sketch (conceptual):

```
# Dev against the R that the image provides
docker run --rm -it myorg/galaaz-r43 \
  bin/galaaz-ruby script/analysis.rb

docker run --rm -it myorg/galaaz-r44 \
  bin/galaaz-ruby script/analysis.rb
```

Inside each image, `R --version` differs; NewBridge still speaks the same protocol. For a first taste without building your own Dockerfile, the repo ships **try** images and runners:

```
# From a Galaaz checkout
./docker/try-gstudio/run.sh    # JRuby + gstudio-oriented image
./docker/try-cruby/run.sh      # CRuby try image
```

**Cold-install** proofs (`docker/cold-install`, `docker/cold-install-cruby`) install the gem on a throwaway Ubuntu and compile the gatekeeper — useful when you want “empty machine → working bridge” confidence. Multi-runtime Docker R matrices are still evolving (slow integration coverage); the sketch above is the pattern to build on.

## 6 Moving data: Apache Arrow

Crossing a process boundary means you must be honest about **copies**. Galaaz’s Arrow roadmap has three stages:

| Stage    | Idea                                                                                                                             | Status                                                                            |
|----------|----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>A</b> | Copy columnar data into R;<br>return an R-side<br>Arrow/proxy handle                                                             | Available<br>( <code>from_ruby_batches</code> ,<br><code>table_from</code> , ...) |
| <b>B</b> | Same-machine <b>IPC file</b> :<br>bulk bytes stay off the<br>MsgPack bridge; only a <b>file</b><br><b>path</b> crosses NewBridge | Shipped (two directions: B1<br>and B2)                                            |
| <b>C</b> | Shared-memory bus (true<br>shared RAM)                                                                                           | Future                                                                            |

Do **not** read Stage A as “zero-copy shared RAM.” Stage A is “build the table in R efficiently, then Remote-Control it.” Stage B is still a file/`mmap` handoff on one machine—not Stage C—but it keeps large columns out of the control plane.

### 6.1 Stage B in plain words

**IPC** here means Apache Arrow’s **Inter-Process Communication** file format: a portable columnar dump on disk (often under `/dev/shm` so it lives in RAM-backed tmpfs). Ruby and R never shove megabytes through NewBridge; they agree on a path, then each side’s Arrow stack reads or writes that file.

Stage B splits by **direction**:

- **B1 — Ruby → R (ingest)**. Ruby writes an IPC file (`Galaaz::ArrowIpc.write / write_batches`). R opens it with `R::Arrow.open_ipc(path)` and gets a table proxy for `dplyr` or modeling. After R has materialised the table, Ruby can `Galaaz::ArrowIpc.release(path)` (unlink the scratch file).
- **B2 — R → Ruby (export)**. After analytics in R, `R::Arrow.write_ipc` writes another IPC file and returns its path over the bridge. Ruby reads columns or row hashes with `Galaaz::ArrowIpc.read / read_batches`—handy when the next step is a DB write or an API payload in Ruby.

Needs: R package `arrow`, plus a Ruby writer/reader (CRuby: **red-arrow**; JRuby: Arrow Java JARs). Specs skip when those are missing; the sketch below does the same.

## 6.2 Stage B sketch (B1 ingest + B2 export)

This is the path to prefer for large same-machine handoffs. Ruby threads build row batches; **B1** writes one IPC file and R opens it by path; dplyr summarises in R; **B2** writes the result IPC and Ruby reads row hashes. NewBridge only carries paths.

```
ipc_ok = Galaaz::ArrowIpc.available? &&
  (R::Support.eval(
    "requireNamespace('arrow', quietly=TRUE) && " +
    "requireNamespace('dplyr', quietly=TRUE)" == true)
unless ipc_ok
  puts '(Skip: need Arrow IPC backend + R arrow/dplyr.)'
else
  thread_count = 2
  rows_per_thread = 250
  group_count = 5
  batches = []
  mutex = Mutex.new
  threads = []

  thread_count.times do |tid|
    threads << Thread.new do
      start = tid * rows_per_thread
      local = (start...(start + rows_per_thread)).map do |i|
        {
          id: i,
          grp: "g#{i % group_count}",
          value: (i % 17) + 1,
          weight: ((i % 5) + 1) * 0.5
        }
      end
      mutex.synchronize { batches << local }
    end
  end
  threads.each(&:join)

  paths = []
  begin
    # B1: Ruby → IPC file → R table proxy
    # write_batches wants flat row Hashes (not nested batches)
    in_path = Galaaz::ArrowIpc.write_batches(batches.flatten)
    paths << in_path
    tbl = R::Arrow.open_ipc(in_path)
    puts "R class: #{tbl.rclass}"
    puts "IPC in: #{File.basename(in_path)}"

    grouped = R.dplyr___group_by(tbl, :grp)
    summed = R.dplyr___summarise(
      grouped,
      n: E.n(),
      total: E.sum(:value),
      wsum: E.sum(R[:value] * R[:weight])
    )
  end
end
```

```

# B2: R → IPC file → Ruby row hashes
out_path = R::Arrow.write_ipc(summed)
paths << out_path
rows = Galaaz::ArrowIpc.read_batches(out_path)
puts "IPC out: #{File.basename(out_path)}"
rows.first(5).each do |r|
  puts "#{r[:grp]} n=#{r[:n]} " +
    "total=#{r[:total]} wsum=#{r[:wsum]}"
end

total_n = rows.map { |r| r[:n].to_i }.sum
expect_n = thread_count * rows_per_thread
puts "Sum of n (expect #{expect_n}): #{total_n}"
ensure
  paths.each { |p| Galaaz::ArrowIpc.release(p) }
end
end

```

```

## R class: Table
## IPC in: galaaz_ipc_11334_d14fc78bbc338f23.arrow
## IPC out: galaaz_ipc_11334_fa50051334bf33f2.arrow
## g0 n=100 total=897 wsum=448.5
## g1 n=100 total=895 wsum=895.0
## g2 n=100 total=893 wsum=1339.5
## g3 n=100 total=891 wsum=1782.0
## g4 n=100 total=889 wsum=2222.5
## Sum of n (expect 500): 500

```

What to notice: `write_batches / open_ipc` is **B1**; after `dplyr`, `write_ipc / read_batches` is **B2**. Specs: `arrow_ipc_handoff_spec.rb`, `arrow_ipc_export_spec.rb`. Deeper notes: `Documentation/ROADMAP_ARROW_RUBY_R.md`.

### 6.3 Stage A (still useful)

Stage A copies batches straight into R with `R::Arrow.from_ruby_batches`—no IPC file. Fine for modest tables or when you do not have a Ruby Arrow writer installed. Same threaded shape as above, one call instead of B1:

```

arrow_ok = R::Support.eval(
  "requireNamespace('arrow', quietly=TRUE) && " +
  "requireNamespace('dplyr', quietly=TRUE)")
unless arrow_ok == true
  puts '(Skip: need arrow + dplyr in R.)'
else
  batches = [
    [{ id: 1, grp: 'a', value: 10, weight: 1.0 },
     { id: 2, grp: 'b', value: 20, weight: 0.5 }],
    [{ id: 3, grp: 'a', value: 30, weight: 1.5 }]
  ]
  tbl = R::Arrow.from_ruby_batches(batches)
  puts "R class: #{tbl.rclass}"
  grouped = R.dplyr___group_by(tbl, :grp)
  summarised = R.dplyr___summarise(

```

```

    grouped, n: E.n(), total: E.sum(:value))
  out = R.dplyr__collect(summarised)
  puts R.as__data__frame(out)
end

```

```

## R class: Table
##   grp n total
## 1   a 2    40
## 2   b 1    20

```

Larger Stage A demos live in `arrow_from_ruby_batches_spec.rb` and `arrow_large_pipeline_spec.rb` (under `specs/` / `slow-specs/`).

## 7 Keeping the bridge free

Two different “don’t block” stories:

- **R::Async** / **R.eval\_r\_async** — free a **Ruby** thread while the **same** bridge R process works. Good for short/medium calls when you still want one R session.
- **R::Job** — run heavy work in a **child Rscript**. The bridge stays free for other traffic. Package installs (`R.install_and_loads`) use this path so `make/gcc` do not wedge the gatekeeper.

```

begin
  coef = R::Job.eval(<<~R) { |job| job.load_rds }
    fit <- lm(mpg ~ wt, data = mtcars)
    saveRDS(unname(coef(fit)), result_path)
  R
  puts coef
rescue => e
  puts e.class.to_s
  e.message.to_s.scan(/.{1,68}/).each { |line| puts line }
end

```

```
## [1] 37.285126 -5.344472
```

Rule of thumb: short interactive analytics on the bridge; installs and multi-minute fits in **R::Job**. Details and timeouts (`GALAAZ_INSTALL_TIMEOUT_SEC`, `GALAAZ_JOBS_DIR`) are in the manual.

## 8 What stayed the same

Galaaz 2.0 is a new **runtime**, not a new plotting language. These still apply:

- **ggplot2** from Ruby — see *Ruby Plotting with Galaaz / Beautiful Ruby Plots*.
- **dplyr** / **NSE** — see *Non Standard Evaluation in dplyr with Galaaz*.
- **Literate docs** — **gKnit** still wraps knitr so Ruby chunks render next to R (HTML/PDF).
- **Classes, modules, procs** — the *oh my* S4 comparison still teaches structure; unwrap and bridge details follow 2.0 rules.

R-on-Rails (Ruby web + R science) is the product framing: Rails (or any Ruby app) orchestrates; GNU R computes.

## 9 Conclusion

**Galaaz 2.0** bets on **boring, powerful defaults**: standard Ruby engines, standard GNU R, a clear process bridge, containers to pin R, and Arrow-shaped paths for larger tables — with zero-copy shared memory still on the roadmap. The Graal-era prototype proved the *idea*; 2.0 makes the *ecosystem* reachable.

### 9.1 Further reading

- Manual (GitHub Pages): <https://rbotafogo.github.io/galaaz/>
- Arrow roadmap: `Documentation/ROADMAP_ARROW_RUBY_R.md`
- Repo blogs under `blogs/` (ggplot, dplyr, gKnit, oh my, manual)
- Try images: `docker/try-gstudio`, `docker/try-cruby`
- Older Graal-era plot narrative (historical): search “Ruby Plotting with Galaaz in GraalVM” on Towards Data Science